EcoLoop Smart Waste Management System

UML Diagrams and Documentation

System Architecture & Role-Based Workflows

Date: May 2026
Version: 1.0

1. System Architecture Diagram

System Architecture Overview

The EcoLoop system is built on a three-tier architecture:

• Frontend Layer: Angular 16+ standalone components with TypeScript
• Backend Layer: Node.js Express server with ACID-compliant transaction support
• Data Layer: MySQL database with JSON file-based fallback storage

Key Components:
- Frontend: Serves from port 5050, handles UI for all roles
- Backend: Express API on port 5050, provides REST endpoints
- Database: MySQL at 127.0.0.1:3306 with transaction queue management
- Storage: File-based JSON fallback at backend/data/store.json
- ACID Layer: Transaction queue, consistency checks, durability backups

2. Class Diagram

Data Model Classes

Core Entities:

User (id, username, password, name, age, address, phoneNumber, role, barangay, points)
% % Administrator: Manages all accounts, reviews submissions, inspects analytics
% % Collector: Manages pickup routes, notifies residents, marks collections
% % Resident: Submits waste, receives notifications, claims rewards

Submission (id, residentId, residentName, wasteType, weightKg, location, segregationScore, status, pointsEarned, submittedAt, notes)
Pickup (id, routeName, collectorId, collectorName, scheduledDate, truck, zone, loadKg, stops, status)
Reward (id, title, pointsCost, stock, partner, description)
Redemption (id, rewardId, rewardTitle, userId, userName, pointsSpent, redeemedAt, status)
Notification (id, userId, title, message, type, read, createdAt)
Activity (id, type, actor, detail, createdAt)
SmartBin (id, name, location, fillLevel, status, lastCollectedAt, sensorHealth, wasteType)

3. Administrator Workflows

3.1 Administrator Use Case Diagram

Admin Use Cases

Administrator Actions:

1. User Management
   - View all resident, collector, and administrator profiles
   - Manage user roles and permissions

2. Submission Review
   - Review all waste submissions
   - Verify segregation scores
   - Award/deduct eco points

3. Collection Management
   - Monitor collection routes
   - Manage pickup status
   - View pickup completion rates

4. Analytics & Reporting
   - View barangay leaderboard (residents only)
   - Inspect waste type distributions
   - Monitor system ACID compliance
   - Access activity logs

3.2 Administrator Flowchart

Admin Workflow Process

START !’ Login (Admin Credentials)
  !“
SELECT ACTION:
  % % View Dashboard !’ View metrics (Users: 3, Waste: X kg, Points: Y, Pickups: Z)
  % % User Management !’ Select user !’ View profile !’ Update role/info
  % % Review Submissions !’ Filter by status !’ Verify segregation !’ Award points
  % % Monitor Pickups !’ View routes !’ Check status !’ Update completion
  % % Analytics !’ View leaderboard !’ Inspect waste data !’ Check ACID status
  % % Profile !’ View account !’ Logout
  !“
END

3.3 Administrator Activity Diagram

Admin Activities

Activity Flow:

[Login] !’ [Authentication] !’ [Dashboard]
  !“
Decision: Task Type?
  % % [User Management] !’ [Query Users] !’ [Update Profile] !’ [Log Activity]
  % % [Review Waste] !’ [Fetch Submissions] !’ [Verify Segregation] !’ [Award Points] !’ [Update Analytics]
  % % [Monitor Routes] !’ [View Pickups] !’ [Track Status] !’ [Generate Report]
  % % [Analytics] !’ [Calculate Totals] !’ [Process Leaderboard] !’ [Display Charts]
  !“
[Transaction Commit] !’ [Update Database] !’ [Refresh Dashboard] !’ [Logout]

3.4 Administrator Sequence Diagram

Admin API Sequence

Admin !’ Frontend !’ Backend !’ Database

Scenario: Review and Award Points
1. Admin clicks "Review Submissions"
2. Frontend sends GET /api/bootstrap
3. Backend queries database for submissions
4. Database returns filtered submissions with status="Pending"
5. Frontend displays submissions list
6. Admin selects submission, verifies segregation score

7. Admin clicks "Award Points"
8. Frontend sends PATCH /api/submissions/:id/status with status="Approved"
9. Backend executes transaction: update submission + update user points
10. Transaction commits to database
11. Backend returns updated submission and user data
12. Frontend updates UI, logs activity

4. Collector Workflows

4.1 Collector Use Case Diagram

Collector Use Cases

Collector Actions:

1. Route Management
   - View assigned pickup routes
   - Plan collection stops
   - Update route status

2. Resident Notification
   - Notify residents when arriving
   - Send collection confirmations
   - Update collection status

3. Submission Verification
   - Mark waste submissions as collected
   - Record actual collection details
   - Confirm waste weight

4. Performance Tracking
   - View pickup statistics
   - Monitor route efficiency
   - Track completion rates

4.2 Collector Flowchart

Collector Workflow Process

START !’ Login (Collector Credentials)
  !“
SELECT ACTION:
  % % View Routes !’ See assigned pickups !’ Select route
  % % Route Detail !’ View stops !’ Plan sequence !’ Mark ready
  % % Notify Arriving !’ Select resident !’ Send notification !’ Resident receives alert
  % % Mark Collected !’ Select submission !’ Confirm weight !’ Submit collection
  % % Performance !’ View statistics !’ Check completion rates !’ View earnings
  % % Profile !’ View account !’ Logout
  !“
END

4.3 Collector Activity Diagram

Collector Activities

Activity Flow:

[Login] !’ [Authentication] !’ [Dashboard]
  !“
[View Routes] !’ [Select Pickup]
  !“
Decision: Action?
  % % [Notify Arriving] !’ [Create Notification] !’ [Send to Resident] !’ [Update Status]
  % % [Collect Waste] !’ [Verify Submission] !’ [Confirm Details] !’ [Mark Collected] !’ [Award Resident Points]
  % % [View Stats] !’ [Calculate Performance] !’ [Display Metrics]
  !“
[Transaction Commit] !’ [Update Database] !’ [Refresh Dashboard] !’ [Logout]

4.4 Collector Sequence Diagram

Collector API Sequence

Collector !’ Frontend !’ Backend !’ Database

Scenario: Mark Waste as Collected
1. Collector selects waste submission from route
2. Frontend sends GET /api/bootstrap to refresh data
3. Backend returns latest submissions
4. Collector verifies weight and clicks "Mark Collected"
5. Frontend sends PATCH /api/submissions/:id/status with status="Collected"

6. Backend executes transaction:
   - Update submission status
   - Calculate and award resident points
   - Update user points balance
   - Create activity log
7. Transaction commits atomically
8. Backend returns updated submission and analytics
9. Frontend notifies resident via notification
10. Dashboard updates in real-time

5. Resident Workflows

5.1 Resident Use Case Diagram

Resident Use Cases

Resident Actions:

1. Waste Submission
   - Submit waste with type and weight
   - Indicate location
   - Rate segregation quality

2. Collection Notifications
   - Receive arrival notifications
   - Get collection confirmations
   - Track submission status

3. Reward Redemption
   - Earn eco points from submissions
   - View available rewards
   - Redeem points for rewards

4. Leaderboard & Analytics
   - View barangay leaderboard (residents only)
   - Check personal waste statistics
   - Track points earned over time

5.2 Resident Flowchart

Resident Workflow Process

START !’ Login (Resident Credentials)
  !“
SELECT ACTION:
  % % Submit Waste !’ Fill form (type, weight, location, segregation) !’ Submit
  %   % % Backend calculates points !’ Points awarded
  % % Submissions !’ View history !’ Check status !’ See collected items
  % % Notifications !’ View alerts !’ Check collection arrival !’ Mark read
  % % Rewards !’ View available rewards !’ Check point cost !’ Click redeem
  %   % % Check balance !’ Redeem if sufficient !’ Reward claimed
  % % Leaderboard !’ View rankings !’ Check personal position !’ See barangay stats
  % % Profile !’ View account !’ Check total points !’ Logout
  !“
END

5.3 Resident Activity Diagram

Resident Activities

Activity Flow:

[Login] !’ [Authentication] !’ [Dashboard]
  !“
Decision: Task Type?
  % % [Submit Waste] !’ [Fill Details] !’ [Calculate Points] !’ [Store Submission] !’ [Update Points]
  % % [View Notifications] !’ [Filter Messages] !’ [Mark as Read] !’ [Update Inbox]
  % % [Redeem Reward] !’ [Check Balance] !’ [Deduct Points] !’ [Create Redemption] !’ [Update Stock]
  % % [View Leaderboard] !’ [Fetch Rankings] !’ [Display Positions] !’ [Highlight Self]
  !“
[Update Analytics] !’ [Calculate Totals] !’ [Refresh Dashboard] !’ [Logout]

5.4 Resident Sequence Diagram

Resident API Sequence

Resident !’ Frontend !’ Backend !’ Database

Scenario: Submit Waste and Earn Points
1. Resident fills waste submission form (type: organic, weight: 5.2kg, location: barangay, segregation: 85)
2. Frontend validates form
3. Resident clicks "Submit"
4. Frontend sends POST /api/submissions with submission data

5. Backend executes transaction:
   - Validate submission data
   - Calculate points: 5.2kg * 10 points/kg = 52 points
   - Create submission record
   - Update resident user points
   - Create activity log
   - Update analytics
6. Transaction commits atomically
7. Backend returns submission + updated user data
8. Frontend displays confirmation
9. Dashboard updates showing +52 points earned
10. Resident waits for collector notification

6. Database Transaction Model

ACID Transaction Flow

All database operations follow ACID principles:

Atomicity:
- Transaction queue prevents concurrent conflicts
- All-or-nothing: either complete or rollback

Consistency:
- Data validation before writes
- Referential integrity checks
- Balance validation (e.g., sufficient points)

Isolation:
- Transaction queue ensures sequential processing
- No dirty reads or lost updates

Durability:
- File-based backup system (backend/data/backups/)
- Automatic recovery mechanisms
- JSON fallback for MySQL unavailability

Example Critical Operations:
- Award points with submission creation
- Redeem reward with balance deduction
- Mark collection with points transfer

7. API Endpoints Reference

Authentication Endpoints

POST /api/auth/login - User login
POST /api/auth/register - User registration

Bootstrap & Data Endpoints

GET /api/bootstrap - Load all system data
GET /api/health - System health check (ACID status)

Submission Endpoints

POST /api/submissions - Create waste submission
PATCH /api/submissions/:id/status - Update submission status
POST /api/submissions/:id/arrival - Notify resident of arrival

Pickup Endpoints

POST /api/pickups - Create pickup route
PATCH /api/pickups/:id/status - Update pickup status

Notification & Reward Endpoints

GET /api/notifications/:userId - Fetch user notifications
PATCH /api/notifications/:id/read - Mark notification as read
POST /api/redemptions - Redeem reward with points

Smart Bin Endpoints

POST /api/bins - Register new smart bin
PATCH /api/bins/:id/fill - Update bin fill level

8. System Summary

Key Features

' Role-based access control (Administrator, Collector, Resident)
' ACID-compliant transaction management
' Waste submission and point earning system
' Collection route and notification system
' Reward redemption with inventory management
' Real-time analytics and leaderboards
' Smart bin integration with fill level tracking
' Activity logging and audit trail

Technology Stack

Frontend: Angular 16+, TypeScript, RxJS, Standalone Components
Backend: Node.js v24, Express 5, mysql2
Database: MySQL with JSON file-based fallback
Deployment: Unified port (5050) for frontend and backend

Performance Metrics

Frontend Bundle: 1.75 MB (browser)
Backend Bundle: 2.67 MB (server)
Database Connection Pool: 10 concurrent connections
Transaction Queue: FIFO with consistency validation
Build Time: ~14.5 seconds